

Barcode & QR Code Scanner

Computer Vision Project Report
Big Data & AI Engineering · April 17, 2026

1. Introduction

This project implements a complete Computer Vision pipeline for detecting, localizing, and decoding QR codes and barcodes from digital images. The system is built using Python, OpenCV, and the pyzbar library, and is delivered as both a command-line tool and an interactive GUI desktop application.

QR codes (Quick Response codes) are two-dimensional matrix barcodes that encode information as a pattern of black and white squares. Traditional barcodes encode data in a series of parallel lines of varying widths. Both types are widely used in industry, retail, logistics, and digital marketing.

2. Project Objectives

#	Objective	Status
1	Detect QR codes and barcodes from images	■ Complete
2	Localize codes with bounding boxes and polygons	■ Complete
3	Decode the content of each detected code	■ Complete
4	Handle multiple codes in a single image	■ Complete
5	Build an interactive GUI application	■ Complete
6	Save results to TXT and CSV files	■ Complete

3. Computer Vision Concepts

3.1 Image Representation

In OpenCV, every image is stored as a NumPy array with shape (height, width, channels). A color image has 3 channels in BGR order (Blue, Green, Red) — note that OpenCV uses BGR instead of the more common RGB format. Each pixel value is an integer between 0 (black) and 255 (white).

3.2 Grayscale Conversion

Converting an image to grayscale reduces it from 3 channels to 1 channel. This simplifies computation since QR codes and barcodes are inherently black and white. OpenCV uses the formula:

$$\text{Gray} = 0.114 \times B + 0.587 \times G + 0.299 \times R$$

3.3 Thresholding

Thresholding converts a grayscale image to a binary (black/white) image. Otsu's thresholding automatically finds the optimal threshold value by analyzing the image histogram. This isolates the

dark QR pattern from the white background.

3.4 Edge Detection

The Canny edge detection algorithm finds the boundaries of objects in an image. It works in 4 stages: (1) Gaussian blur to reduce noise, (2) gradient calculation using Sobel operators, (3) non-maximum suppression to thin edges, and (4) hysteresis thresholding to finalize edges. In our project, pyzbar uses similar edge analysis internally to locate code boundaries.

3.5 Contour Analysis

Contours are curves that join continuous points along a boundary. OpenCV's `findContours()` traces object boundaries in a binary image. For QR code detection, contours help identify rectangular regions that may contain a code. The polygon points returned by pyzbar represent the precise contour of each detected code.

4. System Architecture

The system is organized into a modular pipeline where each stage transforms the image data toward the final decoded output:

Stage	Operation	OpenCV Function
1. Input	Load image from disk	cv2.imread()
2. Preprocess	Convert to grayscale	cv2.cvtColor()
3. Detect	Find & decode all codes	pyzbar.decode()
4. Localize	Extract polygon points	code.polygon
5. Annotate	Draw boxes and labels	cv2.polylines()
6. Output	Save image + CSV + TXT	cv2.imwrite()

5. Implementation

5.1 Core Scanner

The core scanning function loads an image, converts it to grayscale, and passes it to pyzbar for decoding. Each detected code returns its type (QRCODE, EAN13, CODE128, etc.), decoded data as bytes, a bounding rectangle, and precise polygon points.

```
image = cv2.imread(path) gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY) codes = pyzbar.decode(gray)
```

5.2 Localization

Each detected code provides both a bounding rectangle (x, y, width, height) and a polygon (list of corner points). We draw both for maximum precision — the polygon follows the exact shape of the code, while the rectangle provides a clean bounding box.

```
pts = np.array([[p.x, p.y] for p in polygon], dtype=np.int32) cv2.polylines(image, [pts], True, color, 3)
```

5.3 GUI Application

The GUI is built using Python's built-in Tkinter framework. It provides a dark-themed interface with four main controls: Load Image (opens file browser), Scan Codes (runs the detection pipeline), Save Results (exports to TXT and CSV), and Clear (resets the application state). Results are displayed in a scrollable text panel alongside the annotated image.

6. Results & Discussion

The system successfully detects and decodes both QR codes and standard barcodes from static images. The use of pyzbar provides robust decoding that handles various code types including QRCODE, EAN-13, EAN-8, CODE-128, and CODE-39.

Code Type	Detection	Decoding	Localization
QR Code	■ Yes	■ Yes	■ Polygon + Box

EAN-13	■ Yes	■ Yes	■ Polygon + Box
CODE-128	■ Yes	■ Yes	■ Polygon + Box
EAN-8	■ Yes	■ Yes	■ Polygon + Box
Multiple	■ Yes	■ Yes	■ Per code color

7. Conclusion

This project successfully implements a complete barcode and QR code scanner using Python and OpenCV. The system demonstrates key Computer Vision concepts including image preprocessing, edge detection, contour analysis, and pattern recognition.

The modular architecture separates concerns cleanly: image loading, preprocessing, detection, annotation, and output are each handled independently. This makes the codebase easy to extend — for example, adding real-time webcam scanning or cloud-based result storage.

8. References

- OpenCV Documentation — <https://docs.opencv.org>
- pyzbar Library — <https://github.com/NaturalHistoryMuseum/pyzbar>
- ZBar Barcode Reader — <http://zbar.sourceforge.net>
- NumPy Documentation — <https://numpy.org/doc>
- Canny, J. (1986). A Computational Approach to Edge Detection.